Ejercicio Demostrativo

Nombre tutor

Tiempo estimado: 30/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio – Sistemas de Bases de Datos 2 – Sección]

1. Marco Formativo

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Cooperacion	Así como Redis puede operar en clústeres donde los nodos cooperan para mantener la disponibilidad y la integridad de los datos (por ejemplo, mediante replicación y failover), en el laboratorio se fomenta la colaboración entre los estudiantes para resolver problemas, compartir soluciones y cubrir tareas en caso de dificultades, logrando mejores resultados en equipo.

1.1 Valores

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	El estudiante implementa técnicas de modelado y diseño de bases de datos distribuidas para optimizar la estructura de almacenamiento de información, garantizando la integridad, disponibilidad y consistencia de los datos en entornos distribuidos.
Competencia Específica	El estudiante aplica comandos y buenas prácticas de Redis para gestionar claves, strings, hashes, listas, sets, sorted sets, pub/sub, transacciones y expiración de datos, analizando y proponiendo soluciones para casos prácticos de almacenamiento y procesamiento de información.

2. Palabras Clave

1. **Clave:** Identificador único para cada dato en Redis.
2. **String:** Tipo de dato básico para almacenar valores simples o contadores.
3. **Hash:** Estructura para almacenar objetos con múltiples campos.
4. **List:** Colección ordenada de strings, útil para colas o historiales.
5. **Set:** Colección de elementos únicos, sin orden.

6. **Sorted Set:** Set ordenado por puntuación, ideal para rankings.
7. **Pub/Sub:** Sistema de mensajería en tiempo real.
8. **Transacción:** Ejecución atómica de varios comandos.
9. **TTL:** Tiempo de vida de una clave, para expiración automática.

3. Resumen

Este ejercicio demostrativo guía al estudiante en el uso de Redis como base de datos en memoria, mostrando cómo crear, consultar, modificar y eliminar datos usando diferentes tipos de estructuras. Se exploran comandos para manipular claves, strings, hashes, listas, sets, sorted sets, así como mecanismos de publicación/suscripción, transacciones y expiración de datos. El objetivo es que el estudiante comprenda cómo Redis puede resolver problemas comunes de almacenamiento, procesamiento y comunicación en aplicaciones modernas.

4.Contenido

Descripción del ejercicio

El ejercicio consiste en ejecutar y analizar ejemplos prácticos de comandos Redis, abarcando desde operaciones básicas con claves y strings, hasta el uso de hashes, listas, sets, sorted sets, pub/sub, transacciones y expiración de datos. Se busca que el estudiante entienda el propósito de cada estructura y comando, y cómo aplicarlos en situaciones reales.

Objetivo de la demostración

Demostrar el uso de los principales comandos y estructuras de Redis, para que el estudiante pueda identificar cuándo y cómo utilizar cada uno, optimizando el almacenamiento y acceso a la información.

Recursos necesarios

Especifica los recursos que se utilizarán durante la demostración.

- Docker y docker-compose para levantar Redis.
- Terminal para ejecutar comandos Redis.
- Archivo de ejemplos y guía de comandos.

Desarrollo paso a paso

Paso 1:

Presentar el problema a resolver: cómo almacenar, consultar y modificar datos de manera eficiente y flexible en aplicaciones modernas, utilizando una base de datos en memoria como Redis.

Explicar la importancia de elegir la estructura de datos adecuada (strings, hashes, listas, sets, sorted sets) y de definir convenciones claras para el manejo de claves, facilitando la cooperación entre diferentes partes de una aplicación.

Paso 2:

Analizar la gestión de claves en Redis.

Mostrar cómo una buena nomenclatura y el uso de comandos como SET, GET, EXISTS, KEYS y DEL permiten organizar y acceder a la información de forma eficiente.

Discutir la diferencia entre buenas y malas prácticas en el nombrado de claves y su impacto en la mantenibilidad del sistema.

Paso 3:

Explorar el uso de cadenas de texto (strings) para almacenar valores simples, contadores y fragmentos temporales.

Explicar cómo los comandos INCR, DECR, SETEX y TTL permiten implementar contadores, cachés y datos con expiración automática, optimizando el rendimiento y la gestión de recursos.

Paso 4:

Introducir los hashes como solución para almacenar objetos con múltiples campos, como perfiles de usuario o configuraciones.

Mostrar cómo los comandos HSET, HGETALL, HINCRBY y HDEL facilitan la manipulación eficiente de datos estructurados dentro de una sola clave.

Paso 5:

Analizar el uso de listas para implementar colas, historiales o logs de eventos.

Explicar cómo los comandos LPUSH, RPUSH, LRANGE, LPOP y LTRIM permiten gestionar colecciones ordenadas, mantener un historial limitado y procesar tareas en orden de llegada.

Paso 6:

Presentar los sets como herramienta para manejar colecciones de elementos únicos, como etiquetas o listas de usuarios.

Discutir cómo los comandos SADD, SMEMBERS, SINTER y SUNION permiten realizar operaciones de conjunto, identificar elementos comunes y evitar duplicados.

Paso 7:

Explicar el uso de sorted sets para crear rankings, tablas de posiciones o colas de prioridad.

Mostrar cómo los comandos ZADD, ZREVRANGE, ZSCORE y ZRANGEBYSCORE permiten ordenar elementos por puntuación y obtener los mejores o peores resultados de manera eficiente.

Paso 8:

Describir el mecanismo de publicación/suscripción (pub/sub) para implementar sistemas de mensajería en tiempo real.

Explicar cómo los comandos SUBSCRIBE y PUBLISH permiten que diferentes partes de una aplicación se comuniquen instantáneamente, facilitando la integración de notificaciones o chats.

Paso 9:

Analizar el uso de transacciones y control de concurrencia en Redis.

Mostrar cómo los comandos MULTI, EXEC y WATCH permiten agrupar operaciones atómicas y evitar condiciones de carrera, garantizando la integridad de los datos en escenarios concurrentes.

Paso 10:

Explicar la importancia de la expiración programada de claves (TTL) para gestionar datos temporales, como tokens de verificación o cachés.

Mostrar cómo los comandos EXPIRE, TTL y PERSIST permiten controlar el ciclo de vida de la información y liberar recursos automáticamente.

Tabla 1

Coloca una tabla comparativa, de resultados, parámetros, entradas y salidas, o cualquier elemento que apoye la comprensión del ejercicio.

Elemento	**Descripción**
Claves y nomenclatura	Uso de claves estructuradas como producto:1001:color o usuario:1:nombre para organizar la información y facilitar búsquedas, siguiendo buenas prácticas de nombres para evitar confusiones y mejorar la mantenibilidad.
Strings y contadores	Almacenamiento de valores simples (SET servicio:notificaciones:estado "activo") y uso de comandos como INCR, DECR, SETEX para contadores, cachés y datos temporales, permitiendo operaciones rápidas y eficientes sobre datos numéricos o de texto.
Hashes	Representación de objetos con múltiples campos, como usuarios (HSET usuario:501 nombre "Ana" email "ana@correo.com"), permitiendo acceder, modificar o eliminar campos individuales sin afectar el resto del objeto.

Listas	Implementación de colas, historiales o logs de eventos (LPUSH log:eventos ..., LRANGE log:eventos 0 9), facilitando la gestión de tareas en orden y el almacenamiento de secuencias de acciones.

Resultado esperado

Al finalizar la demostración, el estudiante identificará la estructura básica de un sistema de almacenamiento de datos en memoria con Redis y comprenderá cómo se implementan y controlan los mecanismos fundamentales: organización de claves, uso de diferentes estructuras de datos (strings, hashes, listas, sets, sorted sets), operaciones atómicas, expiración de datos y comunicación en tiempo real (pub/sub).

La evidencia final puede consistir en ejemplos de comandos ejecutados, tablas de resultados, simulaciones de operaciones concurrentes y análisis de escenarios de expiración, actualización y eliminación de datos.

Este resultado permite visualizar de forma concreta una práctica propia del área de bases de datos en memoria y gestión eficiente de la información.

Aplicación práctica

Este ejercicio se relaciona con cursos orientados a bases de datos, sistemas distribuidos, programación y diseño de soluciones, porque muestra el flujo básico de construcción y validación de arquitecturas eficientes para el manejo de datos en Redis, desde la definición de la estructura de claves hasta la comprobación de la integridad y disponibilidad de la información.

También sirve como base para prácticas posteriores sobre integración de Redis con aplicaciones web, desarrollo de sistemas de caché, colas de tareas, sistemas de notificaciones en tiempo real y pruebas de concurrencia y expiración de datos.

Dentro de la carrera, este enfoque complementa otras áreas oficiales, ya que mientras unas se orientan más al análisis o a la teoría computacional, esta práctica se centra directamente en la construcción y control de soluciones funcionales para el manejo seguro y eficiente de la información en aplicaciones modernas.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- **Dificultades encontradas:** Los estudiantes suelen tener dificultad para comprender la diferencia entre los distintos tipos de estructuras de datos de Redis y cuándo utilizar cada una (por ejemplo, cuándo usar un hash en vez de varios strings). También es común la confusión al analizar el comportamiento de las operaciones atómicas (MULTI, EXEC, WATCH) y la expiración automática de claves.

- **Recomendaciones para futuras sesiones:** Explicar primero el flujo general de almacenamiento y recuperación de datos en Redis con ejemplos visuales simples (por ejemplo, guardar y leer un valor, agregar y consultar una lista) antes de introducir operaciones más avanzadas como transacciones, pub/sub o expiración. Desarrollar y analizar cada estructura de datos de manera independiente, mostrando el efecto de cada comando. Utilizar diagramas o tablas para visualizar el estado de las claves antes y después de cada operación.

- **Puntos de mayor dificultad para el estudiante:** Comprender la importancia y el momento adecuado para aplicar operaciones atómicas y mecanismos de expiración. Analizar correctamente los escenarios en los que la concurrencia puede afectar la integridad de los datos, o en los que la expiración automática puede eliminar información relevante si no se controla adecuadamente.

6. Referencias

1. Redis Documentation. (2026). https://redis.io/commands/
2. Redis Documentation. (2026). https://redis.io/docs/data-types/
3. Redis Documentation. (2026). https://redis.io/docs/manual/transactions/
4. Redis Documentation. (2026). https://redis.io/docs/manual/pubsub/
5. Redis Documentation. (2026). https://redis.io/docs/manual/key-expiry/